



LABORATORIO DI RETI INFORMATICHE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA
A.A. 2025/2026

Francesca Righetti
francesca.righetti@unipi.it



ESERCITAZIONE 9

Configurazione di Apache HTTP Server

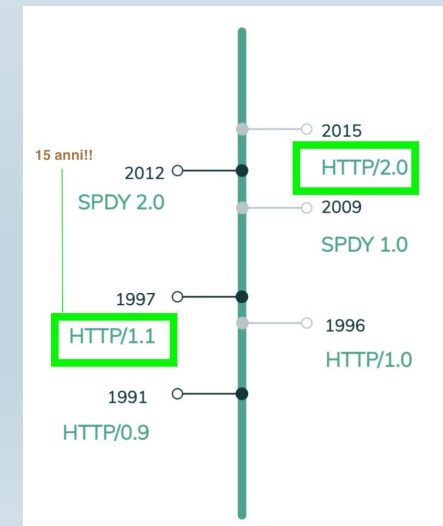
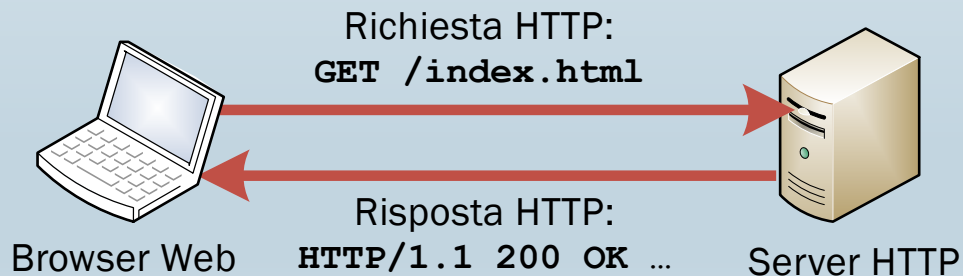
Programma

- Introduzione a HTTP
- Apache HTTP Server
 - Configurazione di base
 - Siti Web: Virtual Host
 - Multi-Processing Modules

HTTP

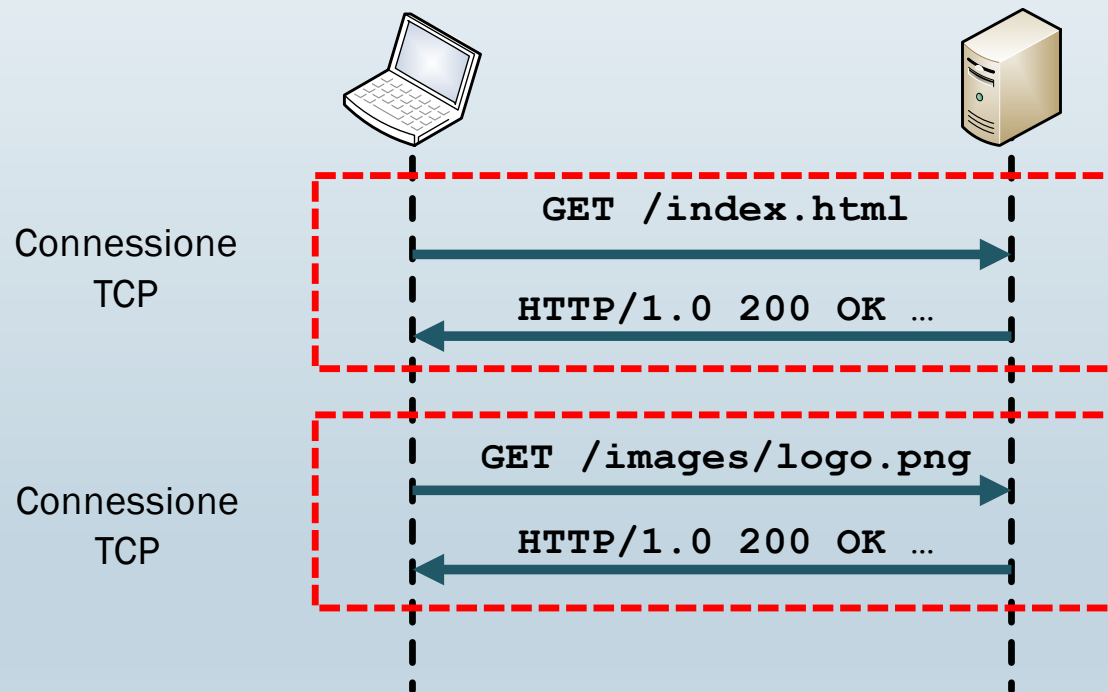
HTTP

- **Hypertext Transfer Protocol (HTTP)** è un protocollo di livello applicazione per lo scambio di ipermedia
 - Il World Wide Web si basa su HTTP
- Architettura **client/server**: il client (browser) chiede un documento e il server HTTP risponde
- Protocollo **stateless**: il server non tiene traccia delle precedenti connessioni. Ogni scambio richiesta/risposta è indipendente dai precedenti.



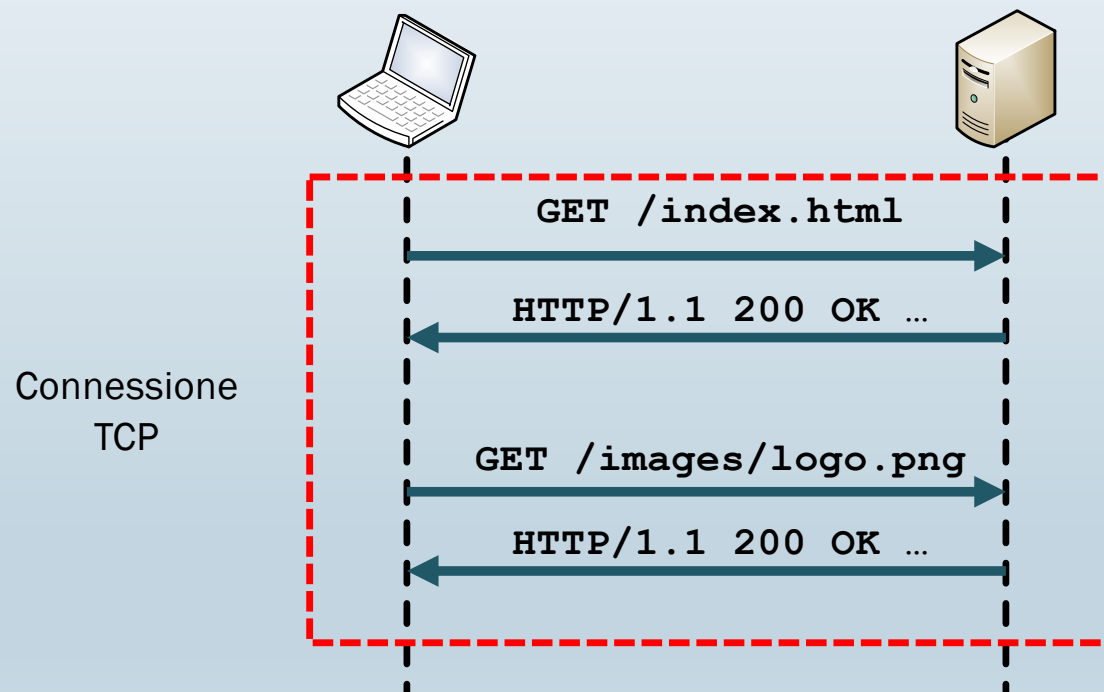
HTTP 1.0 (1996)

- Comunicazioni **non persistenti**: una connessione TCP per ogni scambio richiesta/risposta



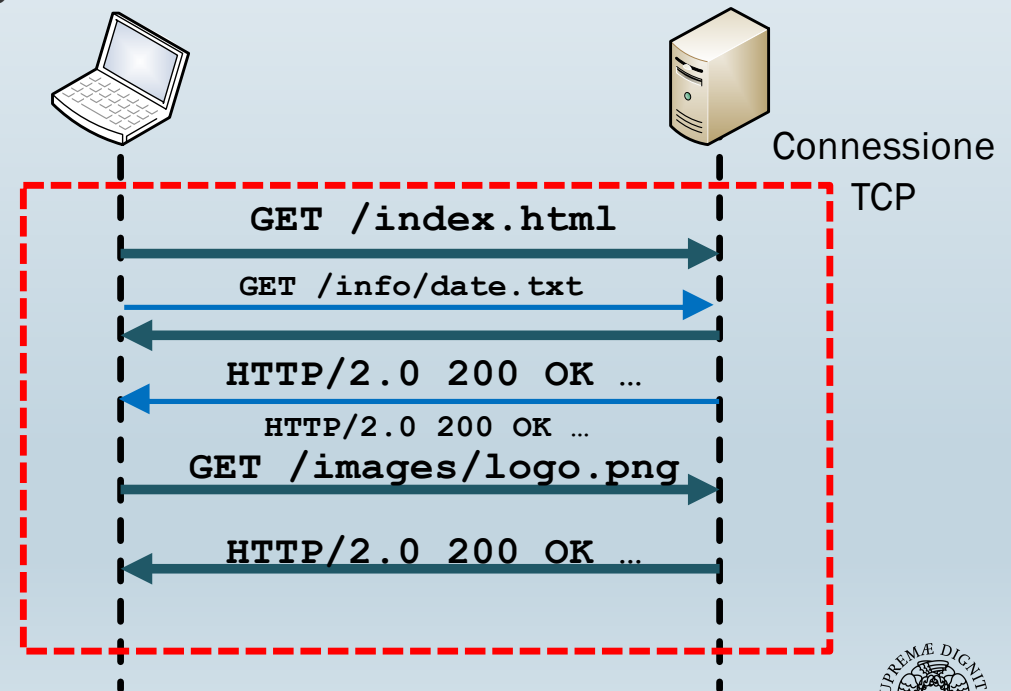
HTTP 1.1 (1997)

- Comunicazioni **persistenti**: una connessione TCP per tutte le richieste consecutive da un client



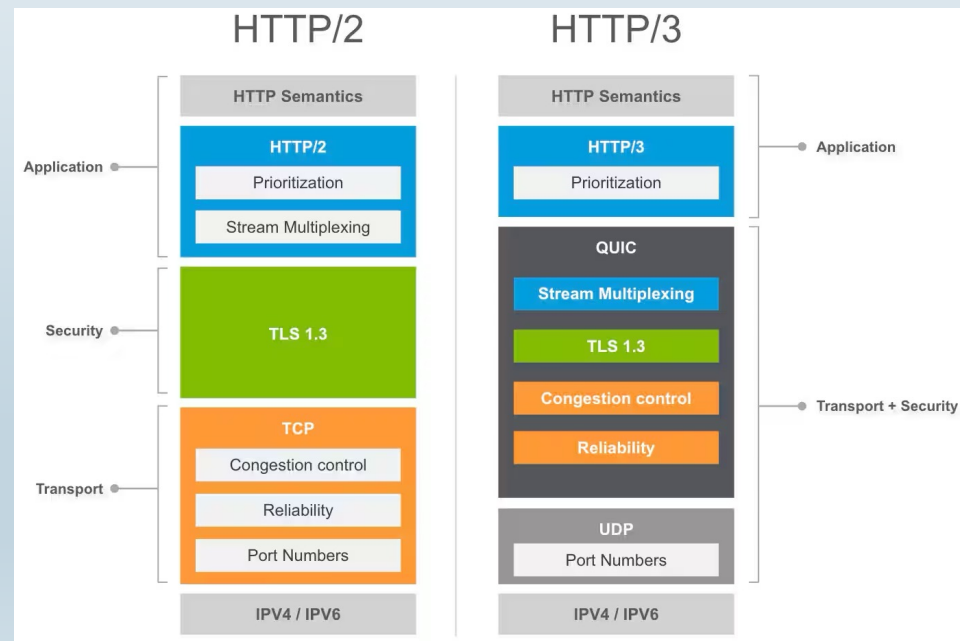
HTTP 2 (2015)

- L'aumento di dispositivi mobili, dell'utilizzo di contenuti multimediali e della richiesta di esperienze web più veloci ed efficienti hanno comportato lo sviluppo di una nuova versione del protocollo
 - **Server Push:** server può inviare risorse in modo proattivo per soddisfare le richieste future del cliente
 - **Multiplexing:** consente più richieste e risposte simultanee per una singola connessione TCP
 - **Priorità del flusso**
 - **Compressione header**
 - **Protocollo binario**



HTTP 3 (2022)

- L'evoluzione del protocollo HTTP ha mirato a migliorare l'efficienza, la sicurezza e la velocità di trasferimento dei dati, passando da una semplice richiesta GET (HTTP/0.9 - 1991) a un protocollo avanzato basato su UDP con multiplexing e crittografia integrata.
- Si basa sul protocollo **QUIC**, che utilizza **UDP** invece di **TCP**
 - Riduce latenza
 - Aumenta sicurezza



Messaggi HTTP

Richiesta:

```
:method: GET
:scheme: https
:authority: www.example.com
:path: /index.html
user-agent: curl/7.79.1
accept: */*
```

Risposta:

```
HTTP/3 200 OK
Date: Wed, 26 Jan 2024 12:34:56 GMT
Server: Apache/2.4.46 (Unix)
Last-Modified: Mon, 10 Jan 2024 10:15:20 GMT
Content-Length: 438
Content-Type: text/html; charset=UTF-8

<qui ci sono i dati>
```

APACHE HTTP SERVER

Apache HTTP Server

- Installare il server nella macchina virtuale. Il server è implementato tramite il programma **apache2**
 - `sudo apt update`
 - `sudo apt upgrade`
 - `sudo apt install apache2`
- Dopo l'installazione, Apache dovrebbe avviarsi automaticamente. Per verificare lo stato di Apache eseguire il comando:
 - `systemctl status apache2`

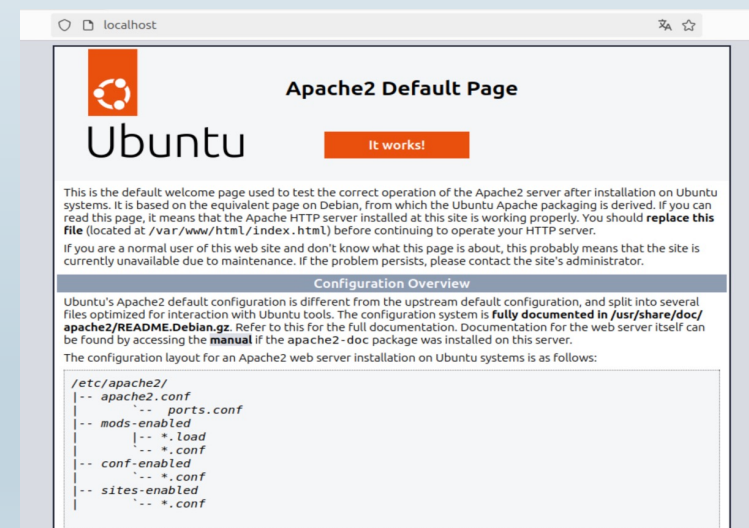
```
ubuntu@ubuntu-reti-so:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: )
   Active: active (running) since Thu 2024-09-26 09:30:59 CEST; 53s ago
     Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 3702 (apache2)
    Tasks: 55 (limit: 4616)
   Memory: 5.5M (peak: 5.7M)
      CPU: 29ms
   CGroup: /system.slice/apache2.service
           └─3702 /usr/sbin/apache2 -k start
             └─3705 /usr/sbin/apache2 -k start
               └─3706 /usr/sbin/apache2 -k start
```

Apache HTTP Server

- Per interagire con il server, si usa il comando **apache2ctl <comando>**, dove i comandi principali sono:
 - start
 - stop
 - restart
 - status
 - configtest
- Si può anche utilizzare il comando **service apache2 <comando>**, dove i comandi principali sono:
 - start
 - stop
 - restart
 - reload
- **Entrambi i comandi richiedono privilegi**

Apache HTTP Server

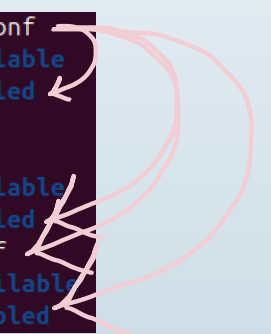
- Quando Apache è in esecuzione si può aprire il browser e accedere al sito Web che mette a disposizione:
 - **http://localhost/**
- La pagina che viene mostrata si trova in:
 - **/var/www/html/index.html**
- Apache è in grado di accettare e servire più richieste contemporaneamente
 - Si può configurare il modello I/O da usare



Apache HTTP Server - Configurazione

- La directory principale è **/etc/apache2**
 - Il file di configurazione principale è **/etc/apache2/apache2.conf**

```
-rw-r--r-- 1 root root 7178 ago 11 13:10 apache2.conf
drwxr-xr-x 2 root root 4096 ott 31 13:37 conf-available
drwxr-xr-x 2 root root 4096 ott 31 13:37 conf-enabled
-rw-r--r-- 1 root root 1782 mar 18 2024 envvars
-rw-r--r-- 1 root root 31063 mar 18 2024 magic
drwxr-xr-x 2 root root 12288 ott 31 13:37 mods-available
drwxr-xr-x 2 root root 4096 ott 31 13:37 mods-enabled
-rw-r--r-- 1 root root 274 mar 18 2024 ports.conf
drwxr-xr-x 2 root root 4096 ott 31 13:37 sites-available
drwxr-xr-x 2 root root 4096 ott 31 13:37 sites-enabled
```



- La configurazione si specifica attraverso delle direttive, eventualmente raggruppate da direttive contenitore

```
# Commento descrittivo
Direttiva1 valore
Direttiva2 valore

<Contenitore valore>
    Direttiva3 valore
    Direttiva4 valore
</Contenitore> # Fine contenitore
```

Apache HTTP Server - Configurazione

- Per "semplificare" la configurazione, Apache usa un **sistema modulare**:
 - Il file di configurazione globale recupera altri pezzi di configurazione da altri file, usando la direttiva **Include**.
 - Esempio: il file **/etc/apache2/ports.conf** contiene le direttive per specificare le porte da usare

```
# File apache2.conf
...
# Include list of ports to listen on
Include ports.conf
...
```

```
# File ports.conf
Listen 80
```

```
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 80

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```


Apache HTTP Server - Configurazione

- Per "semplificare" la configurazione, Apache usa un **sistema modulare**:
 - Il file di configurazione globale recupera altri pezzi di configurazione da altri file, usando la direttiva **Include**.
 - Esempio: il file **/etc/apache2/ports.conf** contiene le direttive per specificare le porte da usare

```
# File apache2.conf
...
# Include list of ports to listen on
Include ports.conf
...
```

```
# File ports.conf
Listen 80
```

```
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 80

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

Apache HTTP Server - Configurazione

- I pezzi di configurazione si mettono nella directory

/etc/apache2/conf-available

- I file vengono **abilitati** con:

```
# a2enconf <nome_file>
```

- Il comando crea un *soft link* nella directory

/etc/apache2/conf-enabled

- Il file di configurazione principale è impostato per includere tutto quello che trova in **conf-enabled**
- **Bisogna riavviare il server per ricaricare la configurazione**
- Un file si disabilita con:

```
# a2disconf <nome_file>
```

Apache HTTP Server – Configurazione

- I pezzi di configurazione che forniscono funzionalità complesse sono detti **moduli** e si trovano nella directory

/etc/apache2/mods-available

- Come per gli altri pezzi di configurazione, si abilitano e disabilitano con:

```
# a2enmod <nome_file>  
# a2dismod <nome_file>
```

- Un modulo abilitato ha un soft link in **mods-enabled**
- **Bisogna riavviare il server per ricaricare la configurazione**

DIRETTIVE GLOBALI

Apache HTTP Server - Direttive Globali

- Le direttive globali sono impostate nel file di configurazione principale di Apache, `apache2.conf`
- Configurazioni generali che vengono applicate al server Apache nel suo complesso, indipendentemente dai siti web o dai virtual host ospitati
 - impostazioni di sicurezza
 - porte su cui Apache ascolta
 - directory predefinite
 - impostazioni di logging
- Le direttive vengono applicate a tutte le richieste e a tutte le risorse servite dal server, a meno che non vengano sovrascritte a livello di virtual host o directory.

Apache HTTP Server - Direttiva ServerRoot

```
ServerRoot /etc/apache2
```

<https://httpd.apache.org/docs/2.4/mod/core.html#serverroot>

- **ServerRoot** specifica la directory principale contenente i file di configurazione di Apache
 - Eventuali path relativi specificati nelle altre direttive sono risolti a partire da questa directory
- Questa direttiva è configurata automaticamente all'avvio del servizio da **apache2ctl**
 - Infatti in **apache2.conf** è commentata

Apache HTTP Server - Direttive KeepAlive, KeepAliveTimeout, MaxKeepAliveRequeststst

```
KeepAlive on  
KeepAliveTimeout 5  
MaxKeepAliveRequests 100
```

<https://httpd.apache.org/docs/2.4/mod/core.html#keepalive>

<https://httpd.apache.org/docs/2.4/mod/core.html#keepalivetimeout>

<https://httpd.apache.org/docs/2.4/mod/core.html#maxkeepaliverequests>

- **KeepAlive** specifica se offrire o meno le connessioni persistenti di HTTP 1.1
- **KeepAliveTimeout** specifica quanti secondi attendere la richiesta successiva dal client sulla stessa connessione prima di chiuderla
 - Un valore troppo alto potrebbe bloccare inutilmente un processo che sta servendo un client lento o disconnesso
- **MaxKeepAliveRequests** specifica quante richieste può fare un client sulla stessa connessione prima di chiuderla

Apache HTTP Server - Direttiva Listen

```
Listen 80  
Listen 8080
```

https://httpd.apache.org/docs/2.4/mod/mpm_common.html#listen

- **Listen** specifica le porte che userà Apache per mettersi in ascolto di connessioni
 - È una direttiva obbligatoria, se è assente il server non parte
 - È possibile specificare più porte
- Questa direttiva è presente in **/etc/apache2/ports.conf**
 - Incluso da **apache2.conf**

Apache HTTP Server - Direttiva ErrorLog

```
ErrorLog /var/log/apache2/error.log
```

<https://httpd.apache.org/docs/2.4/mod/core.html#errorlog>

<https://httpd.apache.org/docs/2.4/mod/core.html#loglevel>

<https://httpd.apache.org/docs/2.4/mod/core.html#errorlogformat>

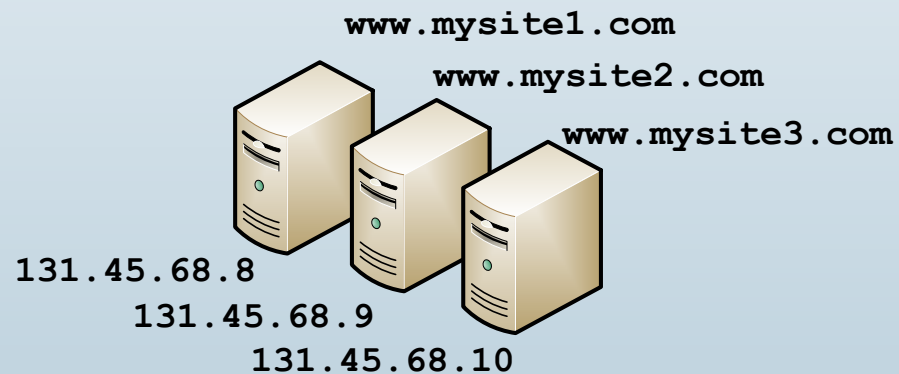
- **ErrorLog** specifica il file di log degli errori
- Il formato e la verbosità dei messaggi possono essere definiti con le altre direttive **ErrorLogFormat** e **LogLevel**

DIRETTIVE PER I SITI WEB

Virtual host

Apache HTTP Server - Virtual Host

- Nel caso più semplice, **un** server Web è in esecuzione su **una** macchina con **un unico** indirizzo IP e offre **un solo** sito Web
- Per ogni sito ci vuole quindi una macchina con un indirizzo IP
- Elevata richiesta di risorse



Apache HTTP Server - Virtual Host

- Con il (***name based*** [Vs *IP-based*]) Virtual Hosting, si possono configurare **più siti sullo stesso server Web**, sulla stessa macchina, con lo stesso indirizzo IP
- Il server Web discrimina le richieste dei client in base al campo **Host** della richiesta HTTP:

```
GET /index.html HTTP/1.1  
Host: www.mysite1.com  
User-Agent: Mozilla/5.0 ...  
Connection: Keep-Alive
```

www.mysite1.com
www.mysite2.com
www.mysite3.com



131.45.68.8

Apache HTTP Server - Virtual Host

- Come per pezzi di configurazione e moduli, i siti/virtual host si mettono in **/etc/apache2/sites-available**

- Si abilitano e disabilitano con

```
# a2ensite <nome_file>  
# a2dissite <nome_file>
```

- Un sito abilitato ha un soft link in **sites-enabled**
- **Bisogna riavviare il server per ricaricare la configurazione**

Apache HTTP Server - Direttiva Virtual Host

```
<VirtualHost ip:porta>  
...  
</VirtualHost>
```

- **VirtualHost** è un contenitore che definisce un virtual host
- Come valore, necessita di indirizzo IP e porta reali, possiamo lasciare *:80

<http://httpd.apache.org/docs/2.4/mod/core.html#virtualhost>

Apache HTTP Server - Direttiva Virtual Host

- Anche nel caso più semplice, Apache ha un Virtual Host di default abilitato in
/etc/apache2/sites-available/000-default.conf

<http://httpd.apache.org/docs/2.4/mod/core.html#virtualhost>

```
<VirtualHost *:80>
    #ServerName www.example.com
    ServerAdmin webmaster@localhost
    ...
    DocumentRoot /var/www/html
</VirtualHost>
```

Indirizzo email dell'amministratore del sito
Per eventuali contatti per malfunzionamenti

Apache HTTP Server - ServerName

```
ServerName www.example.com
```

- **ServerName** specifica il nome simbolico del sito
- È indispensabile per consentire al server di discriminare le richieste dei client

<http://httpd.apache.org/docs/2.4/mod/core.html#servername>

Apache HTTP Server – Direttiva DocumentRoot

```
DocumentRoot /var/www/html
```

- **DocumentRoot** specifica la directory che contiene i file del sito, cioè quelli che vengono messi a disposizione dei client
- **Attenzione alla differenza tra ServerRoot e DocumentRoot**

<http://httpd.apache.org/docs/2.4/mod/core.html#documentroot>

MODULI UTILI

Apache HTTP Server - Direttiva DirectoryIndex

```
DirectoryIndex index.html
```

- Il modulo **dir** specifica, tramite la direttiva **DirectoryIndex**, il file da prelevare in caso non venga specificato dal client
- Fa sì che l'utente che digita **www.mysite.com** riceva in realtà il file **www.mysite.com/index.html**
- È abilitato di default, si trova in

/etc/apache2/mods-available/dir.conf

https://httpd.apache.org/docs/current/mod/mod_dir.html#directoryindex

Apache HTTP Server - Direttiva UserDir

```
UserDir public_html
```

- Il modulo **userdir** consente, tramite la direttiva **UserDir**, ai singoli utenti di mettere i propri file nella cartella specificata e renderli accessibili tramite l'indirizzo

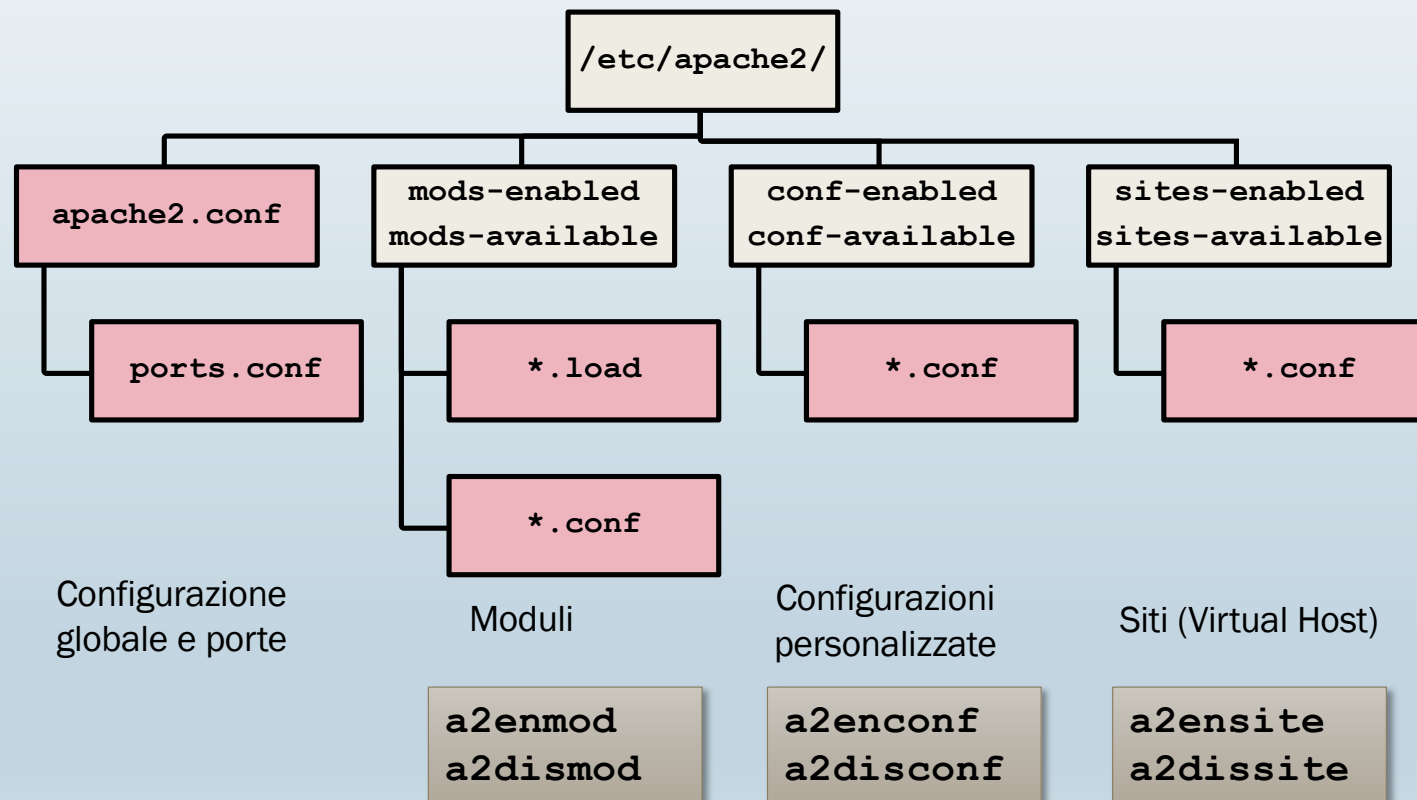
`www.mysite.com/~username/`

- **Non** è abilitato di default, si trova in

`/etc/apache2/mods-available/userdir.conf`

http://httpd.apache.org/docs/2.4/en/mod/mod_userdir.html#userdir

File di configurazione



MULTI-PROCESSING MODULES

Multi-Processing Module

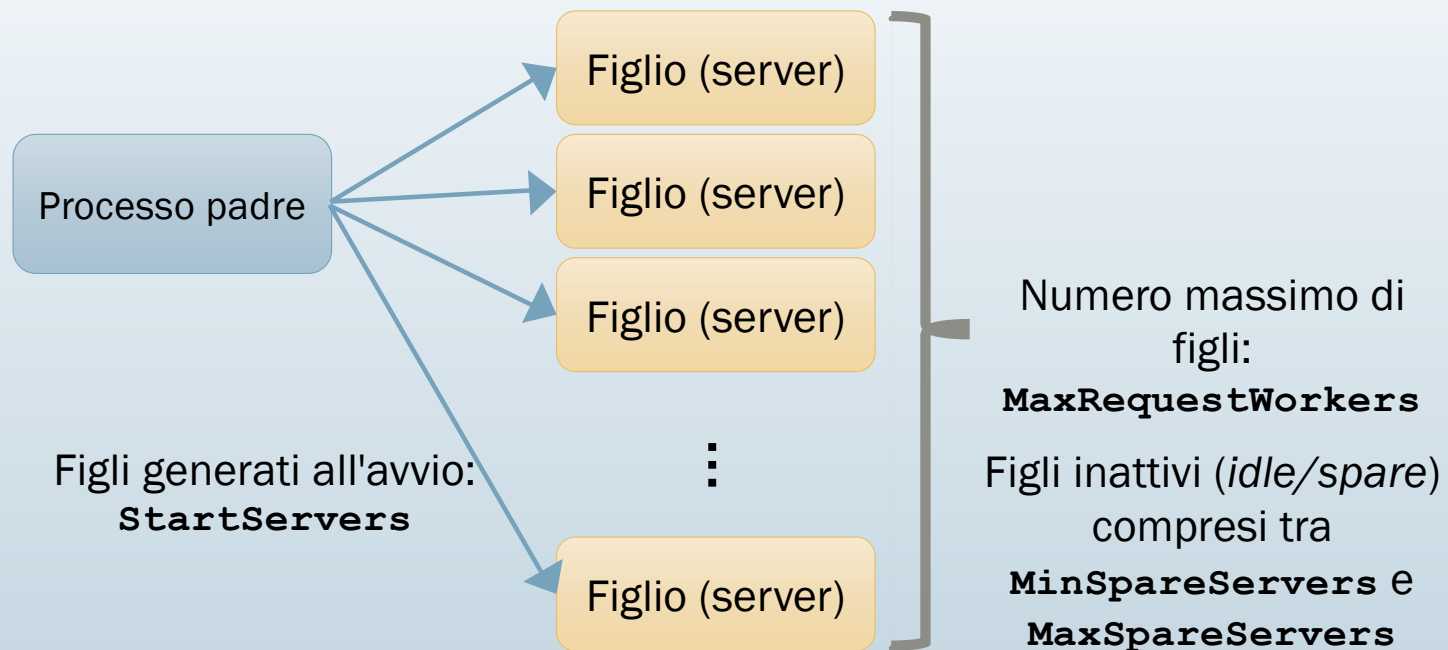
- Apache accetta e serve più richieste contemporaneamente
- Per farlo, ricorre a un Multi-Processing Module (**MPM**), un modulo responsabile di gestire i socket, fare binding delle porte, accettare connessioni e servirle, eventualmente usando processi e thread figli
- Apache supporta diversi MPM, disponibili a seconda del SO e della versione del SO
- Su Unix, si può scegliere tra:
 - **prefork**
 - **worker**
 - **event**

MPM prefork

- Il modulo **prefork** implementa un server **multi-processo, senza thread**.
- All'avvio, un processo padre lancia un certo numero di processi figli (preforking)
 - I figli (workers o servers) restano in ascolto, accettano le connessioni e le servono
 - Dopo ogni connessione servita, un worker torna disponibile
 - Il padre gestisce il pool dei figli, cercando di mantenerne sempre alcuni disponibili
- Il preforking all'avvio e il riuso dei worker evitano l'overhead della **fork()** a ogni connessione.

<https://httpd.apache.org/docs/2.4/mod/prefork.html>

MPM prefork



Ogni figlio viene riciclato per **MaxConnectionsPerChild** connessioni poi viene ucciso, per evitare *memory leak* accidentali

<https://httpd.apache.org/docs/2.4/mod/prefork.html>

MPM prefork

- Vantaggi:

- Massima compatibilità - Alcuni moduli Apache o librerie potrebbero non supportare il multithreading
- Massima stabilità - Un processo che crasha interrompe una sola connessione

- Svantaggi:

- I processi occupano più memoria dei thread
- Bisogna regolare bene le impostazioni: troppi processi inattivi occupano memoria inutilmente, troppo pochi causano più overhead da **fork()** quando ci sono picchi di richieste

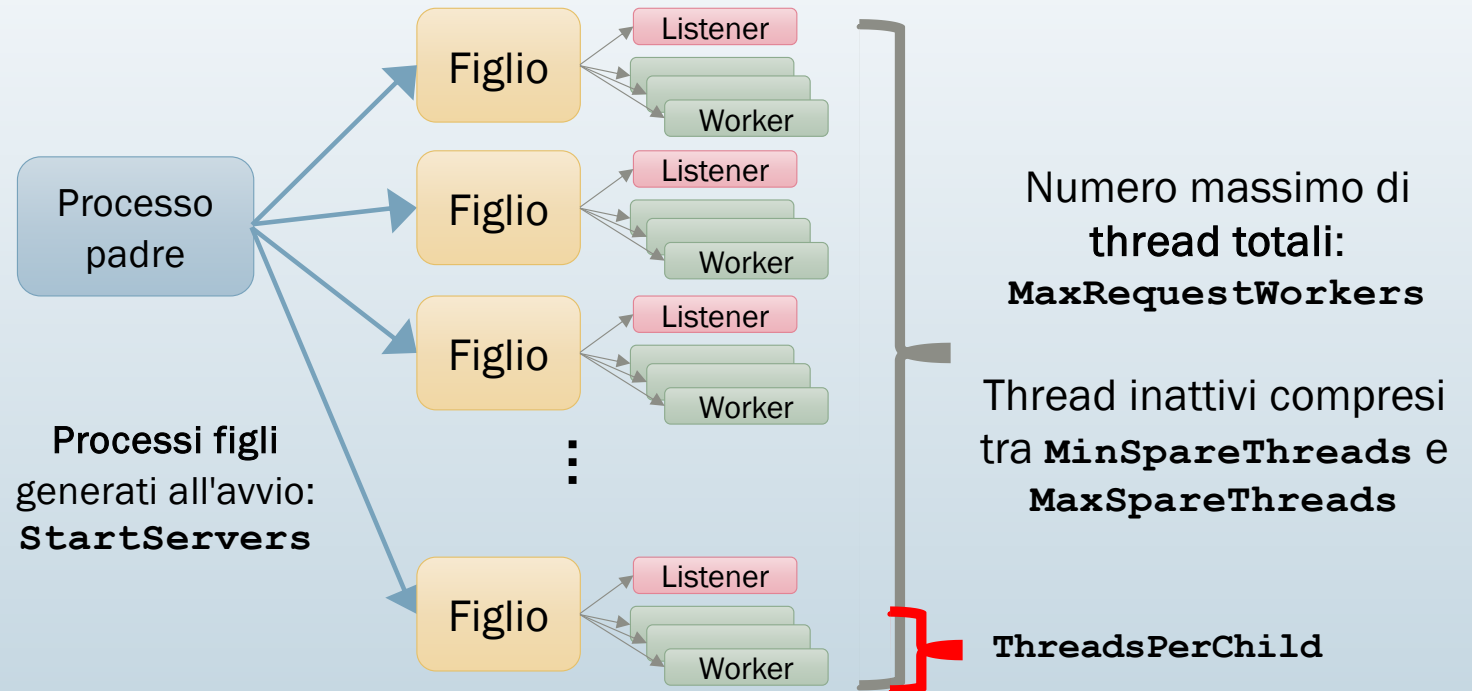
<https://httpd.apache.org/docs/2.4/mod/prefork.html>

MPM worker

- Server **multi-processo e multi-thread**
- Il processo padre genera un certo numero di figli, e ogni figlio genera:
 - un thread listener che accetta e smista le connessioni
 - un certo numero di thread workers che servono
- Overhead ridotto grazie al preforking e risparmio di memoria grazie ai thread

<https://httpd.apache.org/docs/2.4/mod/worker.html>

MPM worker



- Ogni processo figlio viene riciclato per **MaxConnectionsPerChild** connessioni
- Il numero massimo di figli è necessariamente $\text{MaxRequestWorkers} / \text{ThreadsPerChild}$

MPM event

- Versione migliorata di worker
 - **MPM di default** dalla versione 2.4
- Oltre ad accettare le connessioni, il thread listener gestisce le connessioni **temporaneamente inattive**
 - Es 1: Un worker è connesso a un client che sta tardando a inviare una richiesta. Invece di attendere, restituisce il controllo del socket al listener e passa a servire un altro client. Quando il primo client invierà la richiesta, il listener la assegnerà ad un altro worker libero.
 - Es 2: Un worker sta servendo un client con una connessione lenta e il buffer di invio del socket si riempie. Invece di attendere, restituisce il controllo del socket al listener che lo assegnerà ad un altro worker non appena sarà di nuovo scrivibile.
- Aumenta il numero di connessioni servibili contemporaneamente a parità di numero di thread, eliminando i tempi morti
- Dal punto di vista delle direttive, vale quanto detto per worker

Limiti globali

- **ThreadLimit**: limite massimo configurabile per il numero di **thread attivi per processo**
 - *Worker (event)*:

$$\text{ThreadsPerChild} \leq \text{ThreadLimit}$$

- **ServerLimit**: limite massimo configurabile per il numero di **processi figli attivi**
 - *Prefork*:

$$\text{MaxRequestWorkers} \leq \text{ServerLimit}$$

- *Worker (event)*:

$$\text{MaxRequestWorkers} \leq \text{ServerLimit} * \text{ThreadsPerChild}$$

Valori di default

	Prefork	Worker	Event
mods-available/	mpm_prefork.conf	mpm_worker.conf	mpm_event.conf
StartServers	5	2	2
MinSpareServers	5		
MaxSpareServers	10		
MinSpareThreads		25	25
MaxSpareThreads		75	75
ThreadLimit		64	64
ThreadsPerChild		25 (< 64)	25 (< 64)
ServerLimit	256	16	16
MaxRequestWorkers	150 (< 256)	150 (< 25*16)	150 (< 25*16)
MaxConnectionsPerChild	0	0	0

ESERCIZI

Esercizi su Apache HTTP Server

- Verificare con un browser che Apache sia in funzione
- Cambiare la pagina Web iniziale
 - *Eventualmente fare backup della vecchia*
 - *Non serve scrivere codice HTML, basta un file di testo semplice con estensione html*
- Fermare, avviare, riavviare Apache
 - *Per ora ignorare l'eventuale messaggio AH00558*
- Cambiare la porta di ascolto o aggiungerne una
 - *verificare che funzioni*
- Abilitare il modulo UserDir, creare una nuova pagina Web nella cartella utente e testarla
- Verificare la configurazione del Virtual Host di default
 - *Correggere il messaggio di errore AH00558*
- Aggiungere un nuovo Virtual Host con nuove pagine Web
 - *Come faccio a testarlo con il browser?*
- Cambiare il MPM